

SIVRA Canonicalization and Operation Roadmap

9 June 2026

Purpose

This document defines the next canonicalization rules and architecture-neutral IR operations that SIVRA should implement. The operation catalogue is expressed in semantic terms such as `divide`, `square_root`, and `shuffle`. Instruction mnemonics are included only as examples of SIMD instructions that lower to those operations; they are not proposed as IR operation names.

The immediate target is a useful SSE1-oriented catalogue that can later grow to cover wider x86 SIMD instruction sets without changing the basic IR model.

Current Baseline

The built-in operation catalogue currently contains:

- `constant`
- `symbol`
- `memory_load`
- `add`
- `multiply`

The temporary example loader additionally registers `subtract` and `maximum` without reusable semantic metadata.

The canonicalizer currently implements:

- `associative_flattening`
- `identity_elimination`
- `annihilator_collapse`

The existing traits are `associative`, `commutative`, and `idempotent`. Operations may also declare an identity and an annihilator.

Missing Canonicalization Rules

Priority 1: Complete the Existing Trait Model

1. Commutative operand ordering

Sort the operands of commutative operations into one deterministic order.

Examples:

```
add(y, x)          -> add(x, y)
bit_and(mask, x) -> bit_and(x, mask)
```

Requirements:

- A stable structural ordering key for constants, symbols, memory references, and operation nodes.
- Ordering must not depend on transient `node_id` assignment.
- Associative flattening must run first so the complete operand list is sorted.
- Reordering applies only when both the rule and the operation's `commutative` trait are enabled.

2. Idempotent operand deduplication

Remove repeated operands from operations marked `idempotent`.

Examples:

```

bit_and(x, x)    -> x
bit_or(x, x, y)  -> bit_or(x, y)
minimum(x, x)    -> x
maximum(x, x)    -> x

```

Requirements:

- Structural equality or a canonical structural key.
- Associative flattening and commutative ordering should precede this rule.
- A single remaining operand replaces the operation node.

3. Constant folding

Evaluate an operation when all required operands are constants.

Initial scope:

- Arithmetic: `add`, `subtract`, `multiply`, `divide`, `negate`, `square`.
- Bitwise: `bit_and`, `bit_and_not`, `bit_or`, `bit_xor`, `bit_not`.
- Comparisons.
- Conversions with an explicitly defined rounding mode.

Requirements:

- Typed evaluators operating on exact IR constant representations.
- Defined behavior for invalid or exceptional inputs.
- Folding support should belong to operation semantics or an evaluator catalogue, rather than one growing switch in the canonicalizer.

Priority 2: Normalize Arithmetic and Bitwise Forms

4. Negation and subtraction normalization

Choose one canonical representation for subtraction and negation.

Recommended direction:

```

subtract(x, y)    -> add(x, negate(y))
subtract(x, 0)    -> x
negate(negate(x)) -> x
negate(0)         -> 0

```

Using `add` plus `negate` makes associative and commutative addition rules available to expressions originally recovered from subtraction. The rewrite must preserve operand types.

5. Same-operand simplification

Recognize operations whose result is known when operands are structurally equal.

Examples:

```

subtract(x, x) -> 0
bit_xor(x, x)  -> 0
compare_eq(x, x) -> true-mask
compare_ne(x, x) -> false-mask

```

Division should not initially use `divide(x, x) -> 1` because zero and exceptional floating-point values require an explicit policy.

6. Bitwise identity, annihilator, and complement simplification

Extend the existing constant rules to the complete bitwise set.

Examples:

```

bit_and(x, all_bits_set) -> x
bit_and(x, 0)             -> 0
bit_or(x, 0)              -> x
bit_or(x, all_bits_set)  -> all_bits_set
bit_xor(x, 0)             -> x
bit_and(x, bit_not(x))    -> 0

```

```

bit_or(x, bit_not(x))    -> all_bits_set
bit_not(bit_not(x))      -> x

```

Most constant-only cases should reuse identity, annihilator, and constant folding metadata. Complement pairs require a dedicated structural rule.

7. Square recognition

Recover a higher-level square operation:

```
multiply(x, x) -> square(x)
```

This improves decompiler output and gives later rules a direct pattern for squares. `square(x)` may optionally lower back to multiplication in a backend; the canonicalizer should use only one direction.

Priority 3: Cross-Operation Algebra

8. Distributive common-factor extraction

Represent distributivity as a relationship between operations, not as a single operation trait.

Recommended canonical direction:

```

add(multiply(a, b), multiply(a, c))
  -> multiply(a, add(b, c))

```

Factoring is preferred over expansion because it generally reduces graph size and produces more readable recovered expressions.

Requirements:

- Metadata identifying which outer operation distributes over which inner operation.
- Structural matching after commutative ordering.
- Support for factors shared by more than two terms.
- A cost check so factoring is applied only when it reduces or preserves a clearly defined complexity measure.
- Never enable both expansion and factoring as unconditional rewrite directions.

9. Coefficient collection

Combine repeated additive terms after multiplication has a canonical form.

Examples:

```

add(x, x)                                -> multiply(2, x)
add(multiply(2, x), multiply(3, x)) -> multiply(5, x)

```

This depends on constant folding, commutative ordering, and distributive factoring.

10. Minimum and maximum normalization

Implement safe algebraic patterns for `minimum` and `maximum`.

Examples:

```

minimum(x, x)          -> x
maximum(x, x)          -> x
minimum(x, maximum(x, y)) -> x
maximum(x, minimum(x, y)) -> x

```

The operation semantics must state whether these are mathematical operations or preserve architecture-specific NaN and signed-zero operand-selection behavior. Do not mark them commutative or idempotent until that policy is explicit.

Priority 4: Typed and Lane-Structural Rules

11. Comparison normalization

- Canonically order operands when a predicate has a valid swapped equivalent.
- Replace a swapped predicate accordingly, such as less-than becoming greater-than.
- Fold constant comparisons.
- Simplify repeated-operand comparisons only where the predicate's exceptional value behavior permits it.
- Normalize compare-result masks to typed zero or all-bits-set constants.

12. Conversion simplification

Examples:

```
convert(T, constant)          -> typed constant
convert(T, convert(T, x))     -> convert(T, x)
convert(source_type, convert(T, x)) -> x
```

The final example is valid only for proven lossless round trips. Conversion metadata must include source type, result type, signedness, and rounding mode.

13. Shuffle composition

Combine nested lane permutations into one lane map.

Examples:

```
shuffle(shuffle(x, map_a), map_b) -> shuffle(x, compose(map_a, map_b))
shuffle(x, identity_map)          -> x
```

This rule should also recognize when a shuffle is a splat, extract, interleave, or concatenation and replace it with the corresponding clearer operation where that improves output.

14. Extract and insert simplification

Examples:

```
extract_lane(insert_lane(v, i, x), i) -> x
extract_lane(insert_lane(v, i, x), j) -> extract_lane(v, j), when i != j
insert_lane(v, i, extract_lane(v, i)) -> v
extract_lane(splat(x), i)              -> x
```

15. Select simplification

Once comparison masks and `select` exist:

```
select(true_mask, x, y) -> x
select(false_mask, x, y) -> y
select(c, x, x)          -> x
```

Missing IR Operations

Catalogue Design Rules

- Operation names describe semantics, not instruction encodings.
- Packed and scalar instruction forms normally lower to the same operation; the expression result type and lane structure describe the difference.
- Register moves should normally disappear into data flow rather than become arithmetic operations.
- Memory writes and flag writes belong in a future effect representation, not in the pure expression operation catalogue.
- The complete catalogue should be registered with one atomic `register_operations()` batch.

Priority 1: Core Arithmetic and Current Example Coverage

Operation	Arity	Intended semantics	SIMD provenance examples
<code>subtract</code>	2	Ordered subtraction	packed/scalar floating subtraction
<code>divide</code>	2	Ordered division	packed/scalar floating division
<code>minimum</code>	2+	Minimum selection	packed/scalar floating minimum
<code>maximum</code>	2+	Maximum selection	packed/scalar floating maximum
<code>negate</code>	1	Arithmetic sign inversion	commonly recovered from sign-bit manipulation

Operation	Arity	Intended semantics	SIMD provenance examples
<code>square</code>	1	Value multiplied by itself	repeated-operand multiplication

`subtract` and `maximum` should move out of the temporary JSON loader and into the reusable catalogue first. Do not assign a two-sided identity to `subtract` or `divide`; their zero and one simplifications are right-sided.

Priority 2: Floating-Point Unary Operations

Operation	Arity	Intended semantics	SIMD provenance examples
<code>square_root</code>	1	Square root	packed/scalar square-root instructions
<code>reciprocal_approximate</code>	1	Architecture-defined reciprocal approximation	packed/scalar reciprocal instructions
<code>reciprocal_square_root_approximate</code>	1	Architecture-defined reciprocal-square-root approximation	packed/scalar reciprocal-square-root instructions
<code>absolute_value</code>	1	Clear the sign bit under the selected numeric interpretation	common SIMD bit-mask idiom

Approximate reciprocal operations must remain distinct from exact `divide(1, x)` and `divide(1, square_root(x))`.

Priority 3: Bitwise Operations

Operation	Arity	Traits and constants
<code>bit_and</code>	2+	associative, commutative, idempotent; identity all-bits-set; annihilator zero
<code>bit_and_not</code>	2	ordered operation equivalent to <code>bit_and(bit_not(lhs), rhs)</code>
<code>bit_or</code>	2+	associative, commutative, idempotent; identity zero; annihilator all-bits-set
<code>bit_xor</code>	2+	associative, commutative; identity zero
<code>bit_not</code>	1	bitwise complement

These operations cover packed logical SIMD instructions without encoding the register width or opcode in the operation name.

Priority 4: Comparisons

The result should be a typed mask value, normally zero or all bits set per lane.

Operation	Arity	Predicate
<code>compare_equal</code>	2	equal
<code>compare_less_than</code>	2	less than
<code>compare_less_equal</code>	2	less than or equal
<code>compare_unordered</code>	2	at least one unordered operand
<code>compare_not_equal</code>	2	not equal
<code>compare_not_less_than</code>	2	not less than
<code>compare_not_less_equal</code>	2	not less than or equal
<code>compare_ordered</code>	2	both operands ordered

These eight semantic predicates cover the immediate-selected packed and scalar SSE1 comparison family. Scalar compare instructions that write condition flags should lower through a future flag/effect model rather than masquerading as ordinary value-producing expression nodes.

Priority 5: Numeric Conversions

Operation	Arity	Required metadata
<code>convert</code>	1	source type, result type, signedness, active rounding policy
<code>convert_truncate</code>	1	source type, result type, signedness, truncation toward zero
<code>zero_extend</code>	1	source and result integer widths
<code>sign_extend</code>	1	source and result integer widths
<code>bitcast</code>	1	equal-width source and result types

The node's result type supplies part of this information, but conversion validation still needs an explicit operation contract.

Priority 6: Lane and Vector Structure

Operation	Arity	Required parameters
<code>extract_lane</code>	1	lane index
<code>insert_lane</code>	2	lane index
<code>shuffle</code>	1 or 2	result-lane to source-lane map
<code>splat</code>	1	result lane count
<code>interleave_low</code>	2	element width
<code>interleave_high</code>	2	element width
<code>concatenate</code>	2+	source ordering
<code>select</code>	3	mask, true value, false value
<code>pack_mask_bits</code>	1	source lane count and lane width

A general parameterized `shuffle` can represent interleaves and half-register moves, but named higher-level operations remain useful canonical outputs. Lowering may initially create `shuffle`; canonicalization can then recognize `splat`, `interleave_low`, `interleave_high`, or `concatenate`.

Priority 7: SSE1 Integer/MMX-Derived Semantics

Operation	Arity	Intended semantics
<code>average_rounded_unsigned</code>	2	Unsigned rounded average
<code>multiply_high_unsigned</code>	2	High half of an unsigned product
<code>absolute_difference</code>	2	Absolute unsigned difference
<code>sum_absolute_differences</code>	2	Horizontal sum of unsigned absolute differences

The SSE1 MMX extension also uses `minimum`, `maximum`, `extract_lane`, `insert_lane`, `shuffle`, `pack_mask_bits`, and integer extension operations. Signedness and element width belong in types or validated operation parameters, not in opcode-derived operation names.

Required IR and Registry Foundations

The larger operation catalogue should not be added as names alone. The following contracts are needed so malformed nodes cannot be constructed and rules do not infer semantics from strings.

Operation signatures

Add metadata for:

- Minimum and maximum arity.
- Operand and result type constraints.
- Unary, binary, variadic, and ternary operation shapes.
- Whether operands must share the result type.

Directional algebraic constants

The current identity representation is effectively two-sided. Add explicit left and right identities before attaching metadata to ordered operations.

Examples:

- `subtract(x, 0) -> x`, but not `subtract(0, x) -> x`.
- `divide(x, 1) -> x`, but not `divide(1, x) -> x`.

Annihilators may also need sided forms for future ordered operations.

Parameterized nodes

Comparisons can use separate operation definitions per predicate, but lane operations require node parameters. Add a small immutable operation-attribute value capable of storing:

- Lane indices.
- Shuffle lane maps.
- Conversion mode information not derivable from types.

Do not overload `leaf_type_t`; operation attributes describe an operation node, not a leaf expression value.

Structural identity

Commutative ordering, deduplication, distributive factoring, and common subexpression elimination all need a deterministic structural key. Implement:

- A structural comparison independent of `node_id`.
- A structural hash compatible with that comparison.
- Optional hash-consing when adding rebuilt nodes.

Hash-consing is graph construction infrastructure rather than a user-facing algebraic rule, but it should be introduced before the more expensive cross-operation rules.

Operation evaluators

Associate constant evaluators with operations through a private evaluator catalogue or operation semantics extension. Evaluators must:

- Validate arity and types.
- Use exact constant bit representations.
- Return a typed `constant_value`.
- Report unsupported or invalid evaluations without partially rewriting a node.

Recommended Implementation Sequence

Phase 1: Finish the basic catalogue

1. Add operation signature metadata.
2. Add directional identity support.
3. Register `subtract`, `divide`, `minimum`, `maximum`, `negate`, and `square` in the atomic built-in batch.
4. Remove temporary operation registration from the JSON loader.
5. Add registry, metadata, graph-validation, and loader tests.

Phase 2: Finish local canonical forms

1. Add deterministic structural comparison.
2. Implement commutative operand ordering.

3. Implement idempotent operand deduplication.
4. Implement negation/subtraction normalization.
5. Implement same-operand simplification.
6. Implement square recognition.

Phase 3: Add bitwise and constant evaluation

1. Register the five bitwise operations.
2. Add typed operation evaluators.
3. Implement constant folding.
4. Add complement-pair simplification.
5. Test scalar and vector constants independently.

Phase 4: Add comparisons and conversions

1. Register the eight comparison operations.
2. Register conversion operations.
3. Implement comparison normalization and folding.
4. Implement safe conversion folding and round-trip removal.

Phase 5: Add cross-operation algebra

1. Add distributive-relationship metadata.
2. Implement common-factor extraction with a cost check.
3. Implement coefficient collection.
4. Implement safe minimum/maximum absorption patterns.

Phase 6: Add lane structure and the broader SSE1 catalogue

1. Add immutable operation attributes.
2. Register lane and shuffle operations.
3. Implement shuffle composition and extract/insert simplification.
4. Add SSE1 integer/MMX-derived semantic operations.
5. Validate the catalogue against representative SSE1 lowering fixtures.

Completion Criteria

The roadmap is complete when:

- Frontend lowering never registers operation names ad hoc.
- Every supported SIMD expression lowers to an architecture-neutral operation with validated arity, types, and attributes.
- Every operation has explicit algebraic metadata rather than name-based canonicalizer behavior.
- Canonicalization is deterministic across node insertion orders.
- Re-running canonicalization produces an identical canonical structure.
- Each rule has positive, negative, option-disabled, typed, and composition tests.
- The operation catalogue is registered atomically and tested as one coherent set.